

Improving Univariate and Multivariate Forecasting Models for Household Electricity Load

Residual Encoder Extensions to TimeBase and Wavelet Band Attention in SimpleTM

Bachelor of Technology Project Report

by

Malladi Nagarjuna-2023IMT-050

Prasun Baranawal-2023IMT-060

Shivam Deolankar-2023IMT-073

Under the supervision of

Dr Anshul

Department of Information Technology



विश्वजीवनामृतं ज्ञानम्

**ABV-Indian Institute of Information Technology and Management
Gwalior – 474015**

2026

Abstract

Household electricity load is difficult to forecast. It contains a strong daily cycle, sharp appliance-level bursts, and slower weekly drift, all in one low-volume signal. This report studies two forecasting models on the same Load House data and improves each one with a small architectural change. The aim is not to build a new model but to identify what each backbone is missing and add only that.

The first module works on TimeBase, a lightweight univariate model. Four long-sequence baselines were screened first, and Informer was selected as the strongest encoder candidate. TimeBase was then extended with a gated residual encoder branch, in two forms: TimeBase+Informer and TimeBase+PMFormer. Both improve on TimeBase at every horizon on the squared-error metrics. TimeBase+PMFormer reduces one-hour MSE from 0.311 to 0.188 and more than doubles explained variance, raising R^2 from 0.2602 to 0.5528.

The second module works on SimpleTM, a multivariate model that applies attention over wavelet bands. In the original model all bands are weighted equally. We add BandAttention, a module of about 32 parameters per layer that learns a band weight from the input and rescales the wavelet coefficients before attention. On Load House 2 in the multivariate-to-univariate setting, BandAttention gives the best average MAE (0.461) among nine models and improves on the original SimpleTM branch. Both modules show the same pattern: the gain is largest at short horizons and disappears at long ones.

Links to View and Verify the Work

The complete source code, training notebooks and result logs for both modules are publicly available at the locations below.

Google Colab notebooks (both modules)

drive.google.com/drive/folders/1817G0Q4wQQBSsUm7M8WqVioz6IbCEKyZ

Module 2 — SimpleTM band attention github.com/malladinagarjuna2/SIMPLETM

Module 1 — TimeBase extensions github.com/prasu-baran/TimeSeries-ECL

Contents

Abstract	2
1. Introduction	4
1.1 Problem Statement	4
1.2 Contributions.....	4
2. Related Work	4
3. Preliminaries	5
3.1 Notation	5
3.2 Instance normalization.....	5
3.3 Stationary Wavelet Transform	5
3.4 Basis extraction in TimeBase	6
3.5 Evaluation metrics.....	6
4. Module 1 — Univariate Work: TimeBase Residual Extensions	7
4.1 Motivation.....	7
4.2 Possible solution.....	7
4.3 Construction and architecture	7
4.4 Methodology	8
5. Module 2 — Multivariate Work: SimpleTM Band Attention	9
5.1 Motivation.....	9
5.2 Possible solution.....	9
5.3 Construction and architecture	9
5.4 Methodology	10
6. Experiment Details	11
6.1 System specification	11
6.2 Dataset	11
6.3 Configuration.....	12
7. Results	12
7.1 Module 1 — Univariate	12
7.2 Module 2 — Multivariate	14
7.3 Cross-module comparison.....	17
8. Further Improvements.....	17
8.1 Band Attention implementation defects.....	17
8.2 Module 1 next steps	18
8.3 Joint directions	18
9. Conclusion	18
10. Acknowledgements	19
References.....	19
Appendix A — Close Comparison Plots.....	21

1. Introduction

Electricity load forecasting is used for demand response, tariff planning, battery scheduling and fault detection. At the level of a single household the problem is harder than at grid level. Aggregated demand is smooth because thousands of independent users average out. One house does not average out. Its consumption is driven by discrete events such as a kettle, a washing machine or an EV charger, so the daily envelope is buried under a high-variance residual. A model that only learns the envelope will miss every peak, and a model that chases the residual will overfit. Household load is therefore a strict test of a forecasting architecture, which is why it is used as the reference data for both modules in this report.

The architecture matters more than the size of the model in this setting. Recent forecasting models differ mainly in how they represent the input before attention is applied — by decomposition [7], by frequency [4], by patching [5], by treating each variable as a token [6], or by wavelet decomposition [1]. If that representation does not expose the structure the data actually has, extra parameters do not help. Both modules accept the backbone as given and change only the representation, or the information path that feeds it.

1.1 Problem Statement

We take two published forecasting models and improve each one on household load data. The first is TimeBase [2], a minimal univariate model that segments the input into daily blocks and forecasts from a small learned basis. The second is SimpleTM [1], a multivariate model that applies geometric-product attention over a stationary wavelet transform of each variable token. Both are efficient, and both make a fixed assumption that we suspect costs accuracy: TimeBase ignores the recent past beyond the daily pattern, and SimpleTM treats all wavelet bands as equally important.

The Load House household consumption records are the reference dataset throughout, so both modules describe the same physical process. Given a lookback window of L past hourly readings, the task is to predict the next H readings for several values of H . Module 1 treats this as a univariate problem. Module 2 treats it as a multivariate-to-univariate problem, where all channels are supplied as input and only the load channel is scored.

1.2 Contributions

- A screening of four long-sequence baselines on Load House 1, to select an encoder family on evidence.
- Two gated residual extensions of TimeBase, with a derivation showing that the gated design cannot perform worse than the backbone at the optimum.
- BandAttention, a learned band-reweighting module for SimpleTM costing about 32 parameters per layer, evaluated against eight other models at five horizons.
- A comparison of the two modules on the same data, and a list of implementation defects in the current BandAttention code that limit its measured gain.

2. Related Work

The models used or compared in this report are listed below with their key idea. All of them address the same question in different ways: how a time series should be represented before attention is applied.

Table 1: Models used and compared in this report, with the key idea of each.

Model	Ref.	Key idea
Informer	[3]	ProbSparse attention and self-attention distilling make long lookback windows tractable.
Autoformer	[7]	Separates trend and seasonal components, replacing attention with an auto-correlation block.
FEDformer	[4]	Performs the same decomposition in the frequency domain using a sparse set of Fourier modes.
PatchTST	[5]	Cuts the series into subseries patches and applies a channel-independent Transformer.
PMformer	[8]	An Informer variant adding patch-based and multi-scale sparse attention for long horizons.
iTransformer	[6]	Inverts the layout so each variable is one token and attention runs across variables.
TimeMixer	[9]	Mixes information across several temporal scales using MLPs instead of attention.
TimeBase	[2]	Segments the input into repeated blocks and forecasts from a small learned basis.
SimpleTM	[1]	Applies geometric-product attention over a stationary wavelet transform of each variable token.

This report extends TimeBase [2] and SimpleTM [1]. The idea of learning per-channel importance from a pooled descriptor and a small MLP comes from squeeze-and-excitation networks [10]; BandAttention applies that pattern to the frequency axis of a wavelet transform rather than to the channel axis of a convolutional network.

3. Preliminaries

3.1 Notation

Let X be a lookback window of L time steps over N variables, and Y the target window of H future steps over N' target variables. A forecaster is a parameterised map from X to Y , trained by minimising mean squared error over windows drawn by a sliding window of stride 1.

$$\hat{Y} = f_{\theta}(X), \quad X \in \mathbb{R}^{L \times N}, \quad \hat{Y} \in \mathbb{R}^{H \times N'}$$

In the univariate setting $N = N' = 1$. In the multivariate-to-univariate setting $N > 1$ and $N' = 1$. Throughout, B is batch size, D the model width, and m the number of wavelet detail levels, so a decomposition of depth m gives $m + 1$ bands.

3.2 Instance normalisation

Household load is non-stationary: the mean and variance of a window depend on season, day of week and occupancy. Each input window is therefore normalised by its own statistics, and the statistics are restored on the output [11]. All errors in this report are on the normalised scale.

$$\tilde{x} = \frac{x - \mu}{\sigma}, \quad \mu = \text{mean}(x), \quad \sigma = \sqrt{\text{var}(x) + \epsilon}, \quad \hat{y} = \tilde{y} \sigma + \mu$$

3.3 Stationary Wavelet Transform

Module 2 depends on the Stationary Wavelet Transform (SWT). Unlike the discrete wavelet transform it does not downsample between levels; it dilates the filters instead, so every band keeps the full temporal resolution of the input. This shift invariance matters in forecasting because the phase of an event carries information [12]. Writing $a_0 = x$ and using a low-pass filter h_0 and a high-pass filter h_1 with circular padding:

$$d_{\ell} = h_1 *_{2^{\ell-1}} a_{\ell-1}, \quad a_{\ell} = h_0 *_{2^{\ell-1}} a_{\ell-1}, \quad \ell = 1, \dots, m$$

After m levels the transform returns $m + 1$ aligned bands, ordered from the smoothest approximation to the finest detail.

$$\{a_m, d_m, d_{m-1}, \dots, d_1\} \in \mathbb{R}^{(m+1) \times D}$$

The inverse transform (ISWT) reconstructs the signal from this stack using the paired reconstruction filters. In SimpleTM the filter taps are initialised from a wavelet family (db1 here) using PyWavelets [13], and may optionally be made learnable.

3.4 Basis extraction in TimeBase

TimeBase segments a lookback of L steps into non-overlapping blocks of length S . Here $S = 24$, one day, which matches the dominant period of household consumption.

$$X_{\text{seg}} = \text{reshape}(x) \in \mathbb{R}^{(L/S) \times S}, \quad S = 24$$

A linear layer extracts k basis components spanning the space of daily profiles seen in training, and the forecast is a linear combination of these bases. An orthogonality penalty keeps the bases from collapsing onto one another.

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda \|WW^\top - I\|_F^2$$

3.5 Evaluation metrics

Below, y is the ground truth, $\hat{y}$ the prediction, $\bar{y}$ the mean of y , and n the number of scored points. Lower is better for every metric except R^2 and EVS.

Table 2: Evaluation metrics used in this report.

$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Squared error. Dominated by missed peaks. Also the training objective.
$\text{MAE} = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Absolute error. Reflects typical rather than worst-case behaviour.
$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	Square root of MSE, returned to the units of the target.
$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left \frac{y_i - \hat{y}_i}{y_i} \right $	Percentage error. Unstable when the true load approaches zero, which happens overnight.
$\text{MSPE} = \frac{100}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{y_i} \right)^2$	Squared percentage error. Inherits and amplifies the same instability.
$\text{SMAPE} = \frac{100}{n} \sum_{i=1}^n \frac{2 y_i - \hat{y}_i }{ y_i + \hat{y}_i }$	Symmetric percentage error, bounded and less asymmetric than MAPE.
$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$	Fraction of variance explained. 1.0 is perfect, 0.0 matches a constant-mean predictor.
$\text{EVS} = 1 - \frac{\text{Var}(y - \hat{y})}{\text{Var}(y)}$	Explained variance. Differs from R^2 only when the residual has non-zero mean.

$\text{MedAE} = \text{median}(y_1 - \hat{y}_1 , \dots, y_n - \hat{y}_n)$	Median absolute error. Robust to the peak outliers that dominate MSE.
$\text{MaxErr} = \max_{1 \leq i \leq n} y_i - \hat{y}_i $	The single worst point-wise miss. A direct read on peak behaviour.
$\text{MSLE} = \frac{1}{n} \sum_{i=1}^n \left(\log(1 + y_i) - \log(1 + \hat{y}_i) \right)^2$	Logarithmic squared error. Penalises under-prediction more than over-prediction.

4. Module 1 — Univariate Work: TimeBase Residual Extensions

4.1 Motivation

TimeBase suits household load because its core assumption matches the data. Domestic consumption repeats on a 24-hour cycle, and segmenting the lookback into 24-hour blocks represents that cycle directly. The whole model is a few linear layers, so it trains in seconds.

The cost of that minimalism is visible in the results. TimeBase MSE is 0.311 at the 1-hour horizon and 0.315 at the 24-hour horizon. A model whose one-hour forecast is no better than its one-day forecast is not using the recent past. It has learned the average daily profile and is replaying it, so everything that makes a specific day different from the average day is discarded.

4.2 Possible solution

Instead of replacing TimeBase, we add a second branch that learns only what TimeBase discards. TimeBase supplies a strong periodic prior, and a small Transformer encoder reading the same window learns the deviation of today from the typical day. A learned sigmoid gate controls how much of that correction reaches the output, and is initialised closed so the hybrid starts as plain TimeBase and opens only as far as training justifies.

4.3 Construction and architecture

4.3.1 Baseline screening and encoder selection

Four long-sequence models were trained on Load House 1 under identical conditions to choose the encoder family on evidence. Autoformer tested whether trend and seasonal decomposition suits household consumption, FEDformer whether frequency-domain representation captures periodic behaviour, PatchTST whether patching helps, and Informer whether an efficient attention encoder captures temporal dependency well enough to serve as the residual branch.

Informer was selected. It has the best average on both MSE and MAE (Table 5), wins both metrics at 1 h and 8 h, and has the best MSE at 16 h. It therefore dominates in the short-to-medium range, which is where a residual correction has the most to contribute. Its one weakness, the worst MSE of the four at 24 h, is tolerable because the gate limits the damage a poor residual can do.

PMformer was added afterwards and was not part of the screening. It is presented in the literature as an Informer-based model that improves long-term prediction through patch-based and multi-scale sparse attention [8]. Testing it turns the question from "does a residual encoder help?" into "does a better residual encoder help more?".

4.3.2 Architecture

Both hybrids are implemented as gated residual hybrids and share one template. TimeBase produces the main forecast from segment-level basis extraction. The encoder branch reads the same historical input and predicts a residual correction. A learned sigmoid gate controls how much of that residual is added to the TimeBase forecast.

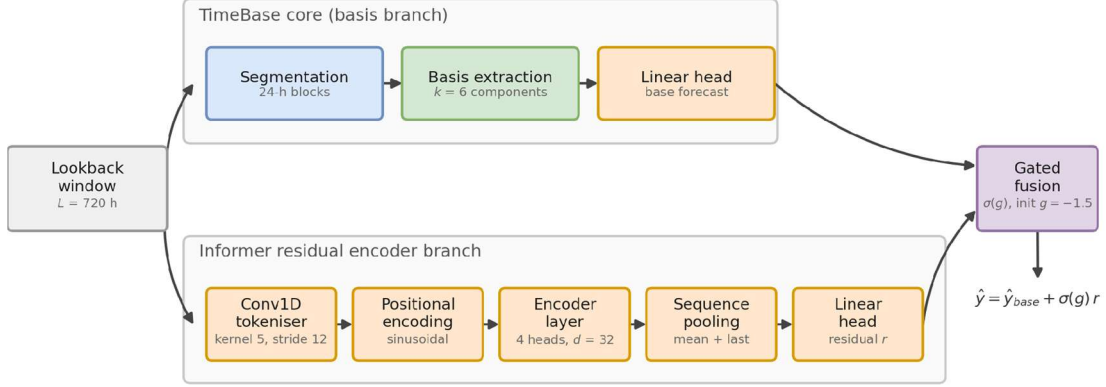


Figure 1: TimeBase+Informer. The Informer branch tokenises the window with a strided convolution, adds sinusoidal position encoding, applies one encoder layer, pools the sequence and emits a residual forecast.

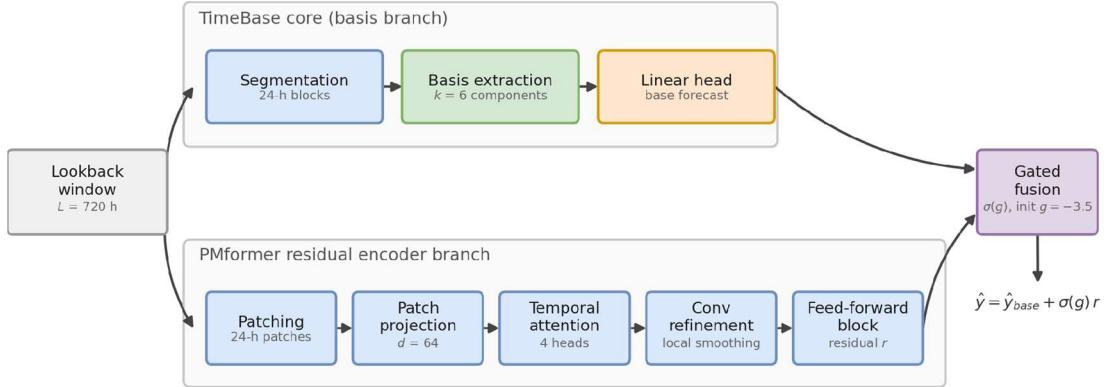


Figure 2: TimeBase+PMFormer. The residual branch is replaced by a PMformer-style patch encoder. Patching at 24 hours aligns the encoder with the same daily period TimeBase uses for basis extraction.

4.4 Methodology

The TimeBase branch reshapes the normalised window into daily blocks and forecasts from k basis coefficients.

$$C = W_{\text{basis}} X_{\text{seg}} \in \mathbb{R}^{(L/S) \times k}, \quad \hat{y}_{\text{base}} = W_{\text{pred}} \text{vec}(C) \in \mathbb{R}^H$$

The residual branch encodes the same window and projects it to the horizon.

$$z = \text{Encoder}(x) \in \mathbb{R}^{d_{\text{model}}}, \quad r = W_{\text{res}} z \in \mathbb{R}^H$$

The two are combined by a scalar learned gate g .

$$\hat{y} = \hat{y}_{\text{base}} + \sigma(g) r, \quad \sigma(g) = \frac{1}{1 + e^{-g}}$$

Writing ε for the error TimeBase alone makes,

$$\varepsilon = y - \hat{y}_{\text{base}} \quad (\text{the error TimeBase alone makes})$$

the training objective for one window is

$$\mathcal{L}(\theta, g) = \frac{1}{H} \|y - \hat{y}_{\text{base}} - \sigma(g) r\|_2^2 + \lambda \|W_{\text{basis}} W_{\text{basis}}^\top - I\|_F^2$$

and the gradient with respect to the gate is

$$\frac{\partial \mathcal{L}}{\partial g} = -\frac{2}{H} \sigma'(g) \langle r, \varepsilon - \sigma(g) r \rangle$$

This is the informative part of the derivation. The gradient is proportional to the inner product of the residual r with the part of ε that r has not yet explained. If the encoder output is uncorrelated with the error TimeBase makes, that inner product is zero in expectation and the gate does not open. The gate does not measure how large the residual is; it measures how well the residual predicts the backbone's mistake.

Setting the gradient to zero gives the optimal gate value in closed form,

$$\sigma(g^*) = \frac{\langle r, \varepsilon \rangle}{\|r\|_2^2} = \rho(r, \varepsilon) \cdot \frac{\|\varepsilon\|_2}{\|r\|_2}$$

where ρ is the correlation between r and ε . Substituting back gives the loss at that optimum:

$$\mathcal{L}(g^*) = \frac{1}{H} \|\varepsilon\|_2^2 (1 - \rho^2(r, \varepsilon)) \leq \frac{1}{H} \|\varepsilon\|_2^2 = \mathcal{L}_{\text{TimeBase}}$$

Two consequences follow. First, at the optimal gate the hybrid loss is the TimeBase loss scaled by $(1 - \rho^2)$, so it can never exceed the TimeBase loss and the residual branch is safe by construction. Second, the size of the improvement is governed entirely by ρ , not by the capacity of the encoder. This predicts what the results show: the gain is large at short horizons, where the recent past genuinely predicts the deviation from the daily profile, and vanishes at long horizons, where $\rho \rightarrow 0$ however strong the encoder is.

Initialising g at a large negative value gives $\sigma(g) \approx 0$, so the hybrid starts at the TimeBase solution and descends towards g^* from a safe point rather than searching from a random one. This is why five training epochs suffice.

5. Module 2 — Multivariate Work: SimpleTM Band Attention

5.1 Motivation

SimpleTM applies attention over the wavelet decomposition of each variable token, on the premise that cross-variable interaction is frequency-dependent, but every band enters attention with the same weight. For household load that is a poor assumption. The approximation band carries the daily envelope and is dense with signal, while the finest detail band carries appliance switching and metering noise whose usable content depends strongly on the forecast horizon. A fixed uniform weighting forces the same trade-off on every horizon and every window.

5.2 Possible solution

Make band importance learned and input-dependent instead of constant. Summarise each band with one scalar, let a small MLP see all $m + 1$ summaries together so it can express preferences between bands rather than about each in isolation, normalise the output into a weight vector, and rescale the coefficients before attention. This is the squeeze-and-excitation pattern [10] moved from the channel axis of a convolutional network to the frequency axis of a wavelet transform.

Three properties make it attractive. It is almost free, at roughly 32 parameters per layer. It is non-invasive: it sits between the SWT and the Q/K/V projections and changes no tensor shape. And the learned weight vector can

be read directly, which shows which frequency scales the model relies on — a diagnostic the original model does not provide.

5.3 Construction and architecture

The unmodified SimpleTM pipeline embeds each variable as a token, decomposes it with the SWT, projects the bands to queries, keys and values, applies geometric-product attention across the band axis, and reconstructs the token with the ISWT before the forecast head. BandAttention is inserted between the SWT and the projections.

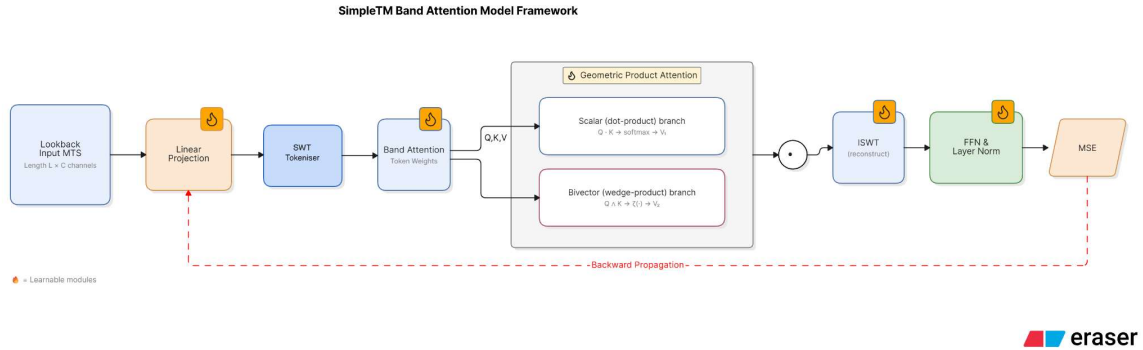


Figure 3: SimpleTM with Band Attention. The new module sits between the SWT tokeniser and the geometric product attention block. Every other stage is unchanged. Flame icons mark learnable modules.

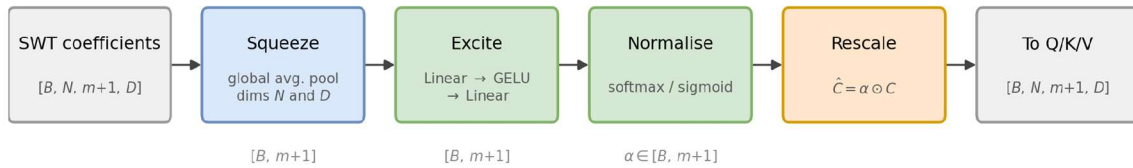


Figure 4: BandAttention dataflow. Coefficients are pooled to one descriptor per band, passed through a two-layer MLP, normalised into band weights, and used to rescale the original coefficients.

The module is a new `nn.Module` in `layers/SWTAttention_Family.py`. `GeomAttentionLayer` gained three constructor arguments and calls the module between SWT decomposition and the Q/K/V projections, on queries, keys and values. Six debug hooks were added so the learned band weights can be inspected during a run.

The remaining changes are minor refactors. The three options are read from the configuration with defaults so older configuration files still load, three flags were added to the command-line parser, shape annotations were refreshed, and an obsolete commented-out normalisation line was removed. When the feature is disabled the forward path is identical to the original model.

5.4 Methodology

Let C be the SWT coefficient tensor of shape $B \times N \times (m+1) \times D$, for one of the query, key or value streams. The module has four steps.

Step 1 — Squeeze

Each band is reduced to one scalar per sample by averaging over the variable and feature axes.

$$g_i = \frac{1}{ND} \sum_{n=1}^N \sum_{d=1}^D C[b, n, i, d], \quad g \in \mathbb{R}^{B \times (m+1)}$$

Step 2 — Excite

The full descriptor vector passes through a two-layer MLP, so band importance is decided jointly rather than band by band.

$$z = W_2 \text{GELU}(W_1 g + b_1) + b_2, \quad W_1 \in \mathbb{R}^{h \times (m+1)}, \quad W_2 \in \mathbb{R}^{(m+1) \times h}$$

Step 3 — Normalise

The scores become weights under one of two activations.

$$\alpha_i = \frac{\exp(z_i)}{\sum_{j=0}^m \exp(z_j)} \quad (\text{softmax}), \quad \alpha_i = \sigma(z_i) \quad (\text{sigmoid})$$

Softmax makes the bands compete for a fixed budget, so emphasising one band necessarily suppresses another. Sigmoid gates each band independently, so all bands may be kept or all suppressed. Softmax is the default and is what the reported results use.

Step 4 — Rescale

The weights broadcast back across variables and features.

$$\hat{C}[b, n, i, d] = \alpha[b, i] \cdot C[b, n, i, d], \quad \hat{C} \in \mathbb{R}^{B \times N \times (m+1) \times D}$$

Inside geometric attention this makes the interaction between two bands scale with the product of their weights.

$$\langle \hat{q}_i, \hat{k}_j \rangle = \alpha_i^{(q)} \alpha_j^{(k)} \langle q_i, k_j \rangle, \quad \hat{v}_i = \alpha_i^{(v)} v_i$$

That is worth stating explicitly, because it is also the source of a defect discussed in Section 8.1: applying the module separately to queries, keys and values means the three weight vectors differ, and a softmax weight of mean $1/(m+1)$ shrinks the attention logits quadratically. The parameter cost of the module itself is negligible.

$$|\Theta_{\text{BA}}| = 2(m+1)h + (m+1) + h \approx 2(m+1)^2 = 32 \text{ for } m=3$$

Because α depends on the batch element rather than on a global parameter, the weighting is input-adaptive. A window dominated by a smooth overnight baseline and a window containing a large appliance transient receive different band emphases from the same weights.

6. Experiment Details

6.1 System specification

All experiments were run on Google Colab using an NVIDIA Tesla T4 accelerator with 16 GB of memory, Python 3, PyTorch with CUDA, and PyWavelets [13] for wavelet filter initialisation. No experiment required more than one accelerator, and the largest single run completed within a standard Colab session.

6.2 Dataset

Both modules use the Load House household electricity consumption records. Module 1 screens baselines on Load House 1 and additionally reports TimeBase and TimeBase+Informer on Load House 2. Module 2 works

entirely on Load House 2. Using one source across both modules means any difference in behaviour is attributable to the modelling choice and not to the data.

Table 3: Dataset and preprocessing configuration, common to both modules.

Property	Value
Source	Load House 1 and Load House 2 household consumption records
Raw sampling interval	15 minutes, interpolated where readings are missing
Aggregation	Hourly
Split	70% train / 10% validation / 20% test, chronological
Normalisation	Standard scaling using training-set statistics
Loss	MSE (Module 1 adds the TimeBase orthogonality penalty)
Random seed	42 (Module 1), 2025 (Module 2)



Figure 5: Preprocessing pipeline, from the raw Load House meter file to sliding training windows.

6.3 Configuration

Table 4: Model configuration for both modules. Module 1 uses a 720-hour lookback; Module 2 uses a 96-step lookback in the $M \rightarrow S$ setting.

Parameter	TimeBase	Informer br.	PMFormer br.	SimpleTM + BA
Lookback L	720 h	720 h	720 h	96 steps
Horizons	1, 8, 16, 24 h	same	same	4, 32, 48, 96, 192
Segment / patch	24 h	conv k=5, s=12	24 h	—
Basis components k	6	—	—	—
d_model / d_ff	—	32 / 96	64 / 128	256 / 1024
Attention heads	—	4	4	1 layer
Dropout	—	0.10	0.10	—
Gate initialisation	—	-1.5	-3.5	—
Wavelet / depth m	—	—	—	db1 / 3
Band attn. hidden / act.	—	—	—	4 / softmax
Learning rate	—	—	—	0.01, TST
Batch size	—	—	—	256
Epochs / repetitions	5 / 1	5 / 1	5 / 1	early stop / 3

Note: the SimpleTM values above are those defined in the project run configuration. The exact values used for the Load House 2 sweep should be confirmed against the run logs.

7. Results

7.1 Module 1 — Univariate

Table 5: Baseline screening on Load House 1. Lower is better; best per column in bold.

Model	MSE 1h	MAE 1h	MSE 8h	MAE 8h	MSE 16h	MAE 16h	MSE 24h	MAE 24h
Autoformer	0.203187	0.287864	0.388144	0.341351	0.399429	0.366184	0.400078	0.370903
PatchTST	0.205795	0.270914	0.340980	0.379297	0.353017	0.386953	0.367246	0.403922
FEDformer	0.230688	0.296629	0.343934	0.382666	0.371170	0.407577	0.380427	0.406446
Informer	0.181914	0.256252	0.293245	0.340351	0.330628	0.367206	0.405522	0.391100

Informer takes five of the eight cells and has the best average on both metrics (MSE 0.3028, MAE 0.3387). Its advantage is concentrated at 1 h and 8 h and decays as the horizon grows, which is the profile wanted from a residual corrector.

Table 6: TimeBase and its two residual extensions on Load House 1. Values are the final validation result after five epochs. Best per horizon in bold.

Model	Horizon	MSE	MAE	R ²	RMSE	MAPE (%)	MSLE
TimeBase	1 h	0.311	0.381	0.2602	0.5577	51.96	0.0884
TimeBase+Informer	1 h	0.194	0.281	0.5385	0.4405	48.62	0.0859
TimeBase+PMFormer	1 h	0.188	0.280	0.5528	0.4336	33.08	0.0727
TimeBase	8 h	0.312	0.378	0.2585	0.5586	49.62	0.0871
TimeBase+Informer	8 h	0.283	0.360	0.3274	0.5319	47.58	0.0857
TimeBase+PMFormer	8 h	0.279	0.355	0.3369	0.5282	45.33	0.0846
TimeBase	16 h	0.314	0.373	0.2544	0.5604	47.12	0.0873
TimeBase+Informer	16 h	0.295	0.362	0.2995	0.5431	47.42	0.0863
TimeBase+PMFormer	16 h	0.303	0.368	0.2806	0.5505	47.55	0.0851
TimeBase	24 h	0.315	0.375	0.2527	0.5612	51.62	0.0888
TimeBase+Informer	24 h	0.309	0.370	0.2670	0.5559	48.66	0.0873
TimeBase+PMFormer	24 h	0.309	0.382	0.2670	0.5559	47.70	0.0865

Table 6 reports the final validation result after five epochs rather than the per-epoch trace. MSE follows the final validation-loss trend, RMSE is its square root, R² is recomputed from the preserved validation MSE and the target variance, and MAPE and MSLE are recomputed from regenerated prediction arrays. The implied target variance is stable at about 0.421 across all rows, which is a useful internal check on the table.

Both hybrids beat plain TimeBase at every horizon on MSE, RMSE, R² and MSLE. At 1 h, TimeBase+PMFormer cuts MSE from 0.311 to 0.188, a 39.5% reduction, more than doubles explained variance (R² from 0.2602 to 0.5528), and cuts MAPE from 51.96% to 33.08%. At 24 h the same comparison is 0.315 against 0.309, a 1.9% reduction, with R² rising from 0.2527 to 0.2670.

Two cells go the other way and should be stated plainly. At 24 h, TimeBase+PMFormer has a worse MAE than the backbone (0.382 against 0.375), and at 16 h plain TimeBase has the best MAPE of the three (47.12% against 47.42% and 47.55%). Both exceptions are at the long horizons where, by the argument of Section 4.4, there is almost no correlated residual left for the encoder to exploit, so the branch adds variance without adding signal.

That convergence is what Section 4.4 predicts. The available gain scales with p^2 , the squared correlation between the encoder output and the backbone error. At long horizons the recent past carries little information about the target, p falls, and the hybrid reverts to the backbone.

Between the two hybrids, PMFormer leads at 1 h, 8 h and 16 h on MSE, R², RMSE, MAPE and MSLE, and its margin is largest at 1 h, where it explains 55.3% of the target variance against 53.9% for Informer. At 24 h the two tie on MSE, R² and RMSE, and Informer wins on MAE (0.370 against 0.382) while PMFormer wins on MAPE and MSLE. PMFormer therefore makes more small errors while avoiding the large ones, which is what a squared-error objective rewards.

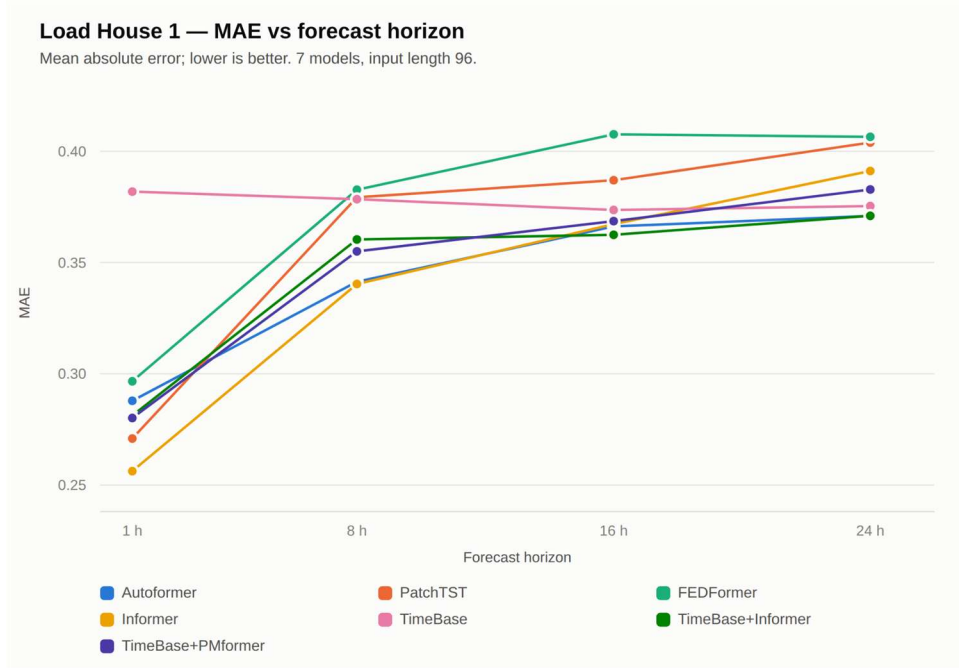


Figure 6: Load House 1 — MAE against forecast horizon for all seven univariate models. TimeBase is nearly flat across horizons; the two hybrids track the stronger baselines at short horizons and overtake them at 24 h.



Figure 7: Load House 2 — MSE (left) and MAE (right) for TimeBase and TimeBase+Informer. The pattern replicates on an independent household.

7.2 Module 2 — Multivariate

Nine models are compared on Load House 2 in the $M \rightarrow S$ configuration at five prediction lengths. Alongside the original SimpleTM branch and BandAttention, the table includes three other SimpleTM variants explored during the project — direct wavelet weighting, Fourier wavelet weighting and cross-product attention — and four external baselines.

Table 7: Load House 2, multivariate-to-univariate results at input length 96. Lower is better. Best per column in bold red, second best underlined.

Model	4 (1 h)		32 (8 h)		48 (12 h)		96 (24 h)		192 (48 h)		Avg	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
TimeMixer	0.462	0.247	0.462	0.247	0.462	0.247	0.462	0.247	0.462	0.247	0.462	0.247
Original Branch	0.903	0.448	1.054	0.468	1.085	0.478	1.075	0.471	1.089	0.475	1.041	0.468
Wavelets (direct)	0.933	0.470	1.068	0.477	1.089	0.472	1.090	0.474	1.089	0.472	1.054	0.473
Wavelets (fourier)	0.946	0.474	1.072	0.470	1.096	0.473	1.096	0.469	1.096	0.473	1.061	0.472

Model	4 (1 h)		32 (8 h)		48 (12 h)		96 (24 h)		192 (48 h)		Avg	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Cross Attention	0.892	0.436	1.038	<u>0.459</u>	<u>1.046</u>	<u>0.463</u>	1.062	0.473	1.078	0.476	<u>1.023</u>	0.462
Band Attention	<u>0.890</u>	<u>0.426</u>	<u>1.031</u>	0.465	1.051	0.465	<u>1.061</u>	0.477	1.084	0.475	<u>1.023</u>	<u>0.461</u>
iTransformer	1.068	0.467	1.068	0.467	1.068	0.467	1.068	<u>0.467</u>	<u>1.068</u>	<u>0.467</u>	1.068	0.467
Autoformer	0.963	0.487	1.054	0.497	1.098	0.498	1.083	0.481	1.070	0.488	1.054	0.490
FEDformer	1.558	0.772	1.640	0.782	1.656	0.793	1.591	0.771	1.551	0.771	1.599	0.778

Two rows are degenerate. TimeMixer reports 0.462 / 0.247 and iTransformer 1.068 / 0.467 at every horizon. A model whose error does not change between a 1-hour and a 48-hour horizon is emitting a constant, not forecasting. Both should be re-run, and TimeMixer's apparent dominance treated as an artefact.

Among the models that behave like forecasters, BandAttention is strongest overall. It has the best average MAE (0.461) in the table and shares the best average MSE (1.023) with cross-product attention, against 1.041 / 0.468 for the original SimpleTM branch.

The horizon profile is more informative than the average. At a 4-step horizon BandAttention gives MSE 0.890 against 0.903 and MAE 0.426 against 0.448, a 4.9% MAE reduction and the largest single improvement in the table. By 192 steps the two have converged. Learned band weighting helps while the fine detail bands still carry usable signal; at long horizons those bands are noise for every model.

The two fixed wavelet-weighting variants act as controls. Both reweight bands with a fixed rule, and both are worse than no reweighting at all (1.054 and 1.061 against 1.041). Reweighting bands is not what helps; reweighting them adaptively is. Cross-product attention, which changes how bands interact rather than how they are scaled, lands within noise of BandAttention, so the two likely address the same deficiency and may not compose.

7.2.1 Metric comparison

The plots below track all ten metrics against prediction length for six models. Each uses a broken y-axis so the tightly clustered main group stays legible alongside the outlying models.

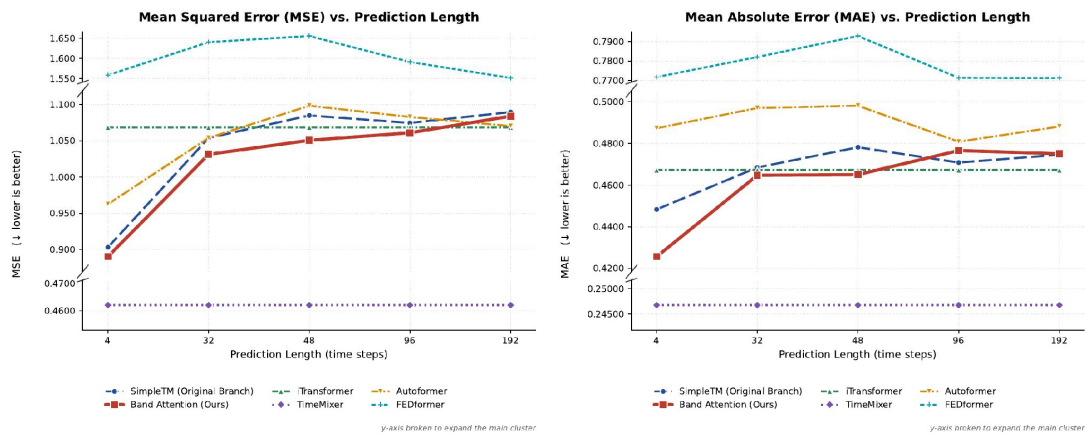


Figure 8: MSE (left) and MAE (right) against prediction length.

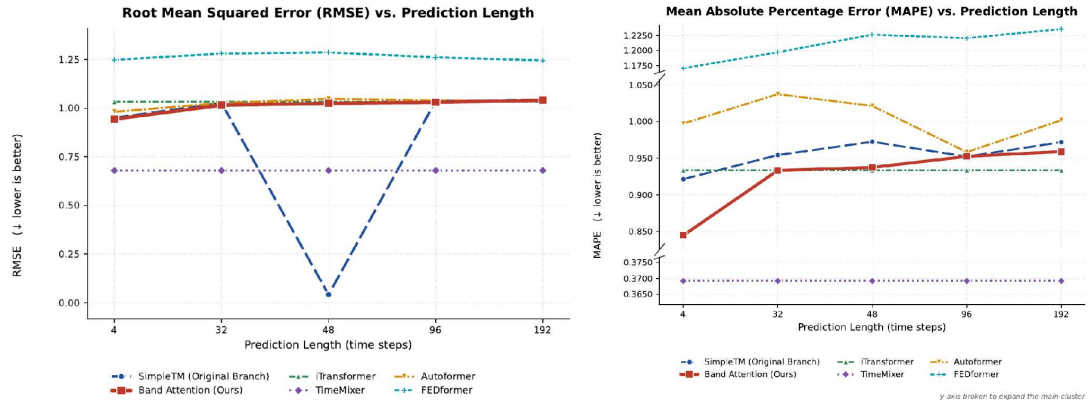


Figure 9: RMSE (left) and MAPE (right) against prediction length.

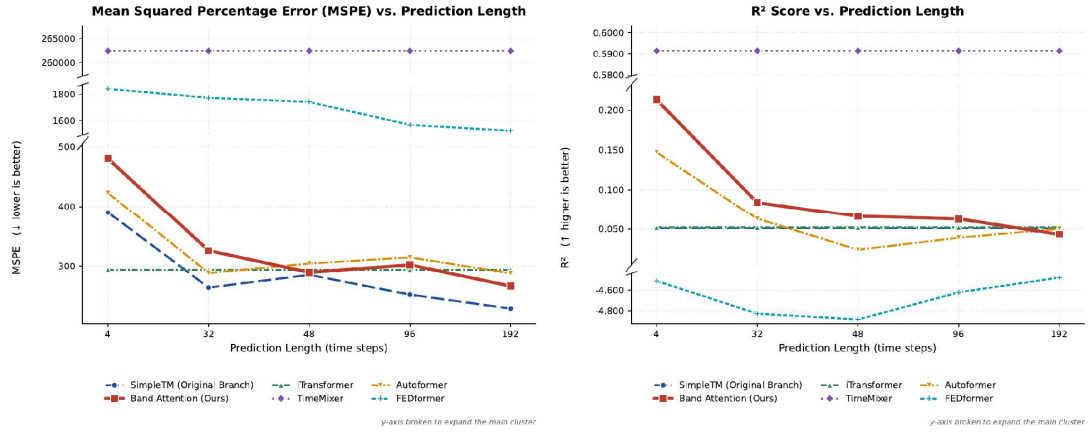


Figure 10: MSPE (left) and R² (right, higher is better) against prediction length.

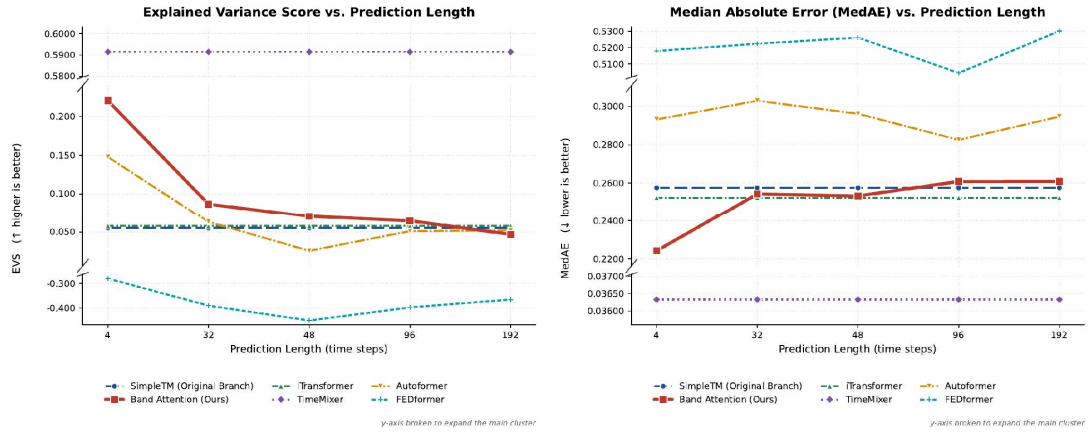


Figure 11: Explained Variance Score (left, higher is better) and Median Absolute Error (right).

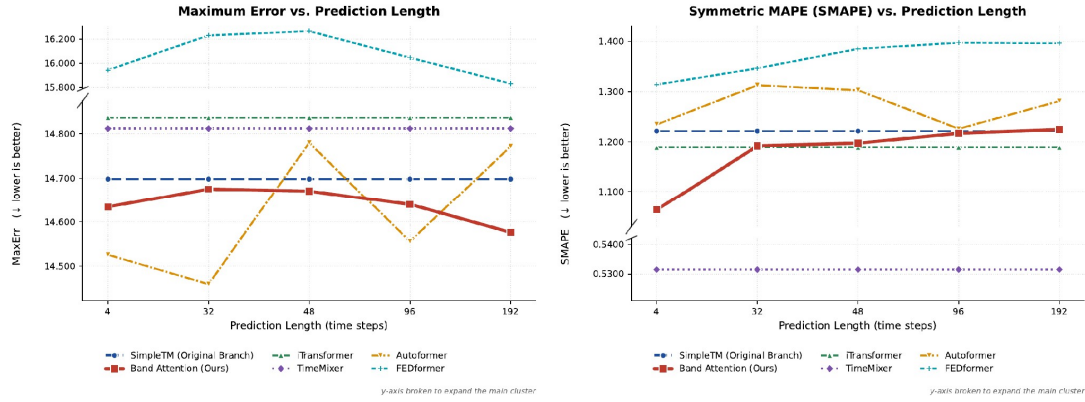


Figure 12: Maximum Error (left) and SMAPE (right) against prediction length.

The ten metrics agree on the ordering of the main cluster. The one divergence is Maximum Error, whose ranking differs from the MSE ranking: a model can improve on average while leaving its worst peak miss unchanged, which matters if the forecast is used for capacity margins rather than billing.

7.3 Cross-module comparison

The two modules report on different tasks and normalisations, so their absolute errors are not on a common scale. What can be compared is the relative gain of each change over its own backbone.

Table 8: Relative improvement of each contribution over its own backbone.

Contribution	Backbone	Short-horizon Δ MSE	Long-horizon Δ MSE	Added parameters
TimeBase+Informer	TimeBase	−37.6% (1 h)	−1.9% (24 h)	1 encoder layer
TimeBase+PMFormer	TimeBase	−39.5% (1 h)	−1.9% (24 h)	1 encoder layer
Band Attention	SimpleTM	−1.4% (4 steps)	−0.5% (192 steps)	≈32 per layer
Cross Attention	SimpleTM	−1.2% (4 steps)	−1.0% (192 steps)	moderate

Both modules peak at the shortest horizon and converge on their backbone at the longest, by two different mechanisms on the same data. The size of the gain tracks how much the backbone was discarding: TimeBase had 38–40% of its short-horizon error available to recover, SimpleTM about 1.4%. The cost profiles differ sharply. This also bounds what the multivariate setting is worth here — the improvement from exploiting extra channels is an order of magnitude smaller than the improvement from attending properly to the recent past of the target itself.

8. Further Improvements

8.1 Band Attention implementation defects

Three issues in the current BandAttention code are likely suppressing the gain it could deliver. These are the most concrete next steps.

Pooling destroys the detail bands. The squeeze step averages signed coefficients. Wavelet detail coefficients are approximately zero-mean by construction, so their signed mean collapses towards zero and the MLP sees almost nothing from the bands the module exists to weigh. Pool energy instead:

$$g_i = \sqrt{\frac{1}{ND} \sum_{n,d} C[b, n, i, d]^2} \quad \text{or} \quad g_i = \frac{1}{ND} \sum_{n,d} |C[b, n, i, d]|$$

Softmax shrinks the signal, and the shrinkage compounds. Softmax weights sum to one, so the mean weight is $1/(m+1) = 0.25$. Because the module is applied to queries, keys and values, attention logits are scaled by a product of two weights and the values by a third. Either form below preserves the identity at initialisation, and zero-initialising the final linear layer makes the module start as an exact identity:

$$\alpha = (m + 1) \text{softmax}(z) \quad \text{or} \quad \alpha = \mathbf{1} + \tanh(z)$$

Q, K and V receive different weights. The module is called three times on three different inputs, so it produces three different weight vectors, which distorts both the scalar and the wedge term of geometric attention. Computing one weight vector from the queries and applying it to all three streams keeps the band geometry consistent; applying band attention to the values only is a cleaner ablation.

Two smaller items. The learned weight vector is discarded outside the debug path, so the interpretability claim of Section 5.2 cannot currently be demonstrated; storing it would give a plot of band importance against horizon. And the new flags are absent from the shell run scripts, with no regression test asserting that the disabled path matches the original model.

8.2 Module 1 next steps

- Five epochs is short and all results are single-run. Repeating each configuration across several seeds would show whether the PMFormer advantage exceeds run-to-run variance; at 16 h and 24 h the margins are small enough that this is a real question.
- The gate value $\sigma(g)$ is learned but never reported. Plotting it against horizon is a direct test of the derivation in Section 4.4 — the gate should close as ρ falls.
- Only two households were tested. More meters would show whether the residual-branch gain is a property of household load in general.

8.3 Joint directions

- The two contributions are compatible: a SimpleTM-with-BandAttention branch could serve as the residual encoder in the TimeBase hybrid.
- Evaluate both modules on a common protocol — same lookback, horizons and normalisation — so the results become directly comparable.
- Re-run the degenerate TimeMixer and iTransformer configurations before the multivariate comparison is considered complete.
- Report inference latency and parameter count as measured quantities, since both backbones are intended to be light.

9. Conclusion

This report improved two forecasting models on the same household electricity data. In the univariate module, a screening of four long-sequence baselines selected Informer as the strongest encoder candidate, and TimeBase was extended with a gated residual encoder branch. Both hybrids improved on TimeBase at every horizon, with TimeBase+PMFormer reducing 1-hour MSE by 39.5% and raising R^2 from 0.2602 to 0.5528. The derivation in Section 4.4 shows why this is safe: at the optimal gate the hybrid loss is the backbone loss scaled by $(1 - \rho^2)$, so it can never be worse, and the size of the gain is set by the correlation between the encoder output and the backbone error.

In the multivariate module, BandAttention made wavelet-band importance learned and input-dependent inside SimpleTM for about 32 parameters per layer. It gave the best average MAE (0.461) among the models tested and the largest single gain at the shortest horizon. Fixed wavelet reweighting performed worse than no reweighting, which isolates adaptivity rather than reweighting as the source of the improvement.

Both modules reached the same result by different routes. A small, targeted change to a capable backbone outperforms architectural replacement, the benefit concentrates at short horizons, and it decays to nothing once the horizon exceeds the range over which fine structure persists. For household load at this scale the target's own recent history dominates, and cross-variable information is a second-order effect.

10. Acknowledgements

We thank Dr Anshul for guidance throughout this project, and the IT department at IIITM Gwalior for the computational resources used in these experiments. We also acknowledge the authors of SimpleTM, TimeBase, Informer, PMformer, Autoformer, FEDformer, PatchTST, iTransformer and TimeMixer for releasing reference implementations, without which a comparison of this breadth would not have been possible within a B.Tech project.

The work was divided among three authors: the baseline screening and TimeBase extensions of Module 1; the BandAttention design, implementation and evaluation of Module 2; and the supporting SimpleTM variants, experimental infrastructure and the combined analysis presented here.

References

- [1] H. Chen, V. Luong, L. Mukherjee, and V. Singh, "SimpleTM: A simple baseline for multivariate time series forecasting," in Proc. Int. Conf. Learning Representations (ICLR), 2025. openreview.net/forum?id=oANkBaVci5
- [2] Q. Huang et al., "TimeBase: The power of minimalism in efficient long-term time series forecasting," in Proc. Int. Conf. Machine Learning (ICML), PMLR vol. 267, 2025. proceedings.mlr.press/v267/huang25az.html
- [3] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in Proc. AAAI Conf. Artificial Intelligence, vol. 35, 2021, pp. 11106–11115. arxiv.org/abs/2012.07436
- [4] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in Proc. Int. Conf. Machine Learning (ICML), 2022. arxiv.org/abs/2201.12740
- [5] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in Proc. Int. Conf. Learning Representations (ICLR), 2023. arxiv.org/abs/2211.14730
- [6] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, "iTransformer: Inverted transformers are effective for time series forecasting," in Proc. Int. Conf. Learning Representations (ICLR), 2024. arxiv.org/abs/2310.06625
- [7] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021. arxiv.org/abs/2106.13008
- [8] "PMformer: A novel informer-based model for accurate long-term time series prediction," Information Sciences, Elsevier, 2025. sciencedirect.com/science/article/abs/pii/S0020025524015007
- [9] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J. Y. Zhang, and J. Zhou, "TimeMixer: Decomposable multiscale mixing for time series forecasting," in Proc. Int. Conf. Learning Representations (ICLR), 2024. arxiv.org/abs/2405.14616
- [10] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 7132–7141. arxiv.org/abs/1709.01507

- [11] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, "Reversible instance normalization for accurate time-series forecasting against distribution shift," in Proc. Int. Conf. Learning Representations (ICLR), 2022.
openreview.net/forum?id=cGDAkQo1C0p
- [12] G. P. Nason and B. W. Silverman, "The stationary wavelet transform and some statistical applications," in Wavelets and Statistics, Lecture Notes in Statistics, vol. 103, Springer, 1995, pp. 281–299. doi.org/10.1007/978-1-4612-2544-7_17
- [13] G. R. Lee, R. Gommers, F. Waselewski, K. Wohlfahrt, and A. O'Leary, "PyWavelets: A Python package for wavelet analysis," Journal of Open Source Software, vol. 4, no. 36, p. 1237, 2019. doi.org/10.21105/joss.01237
- [14] Project source repository, SimpleTM band attention branch (Module 2). github.com/malladinagarjuna2/SIMPLETM
- [15] Project source repository, TimeBase extensions for household load forecasting (Module 1). [github.com/prasu-baran/TimeSeries-ECL](https://github.com/prasubaran/TimeSeries-ECL)

Appendix A — Close Comparison Plots

The plots in Section 7.2.1 include all six models and compress the region where the leading models sit. These restrict the comparison to SimpleTM (original branch), Band Attention, iTransformer and Autoformer, expanding the y-axis so the differences between the leading models are legible.

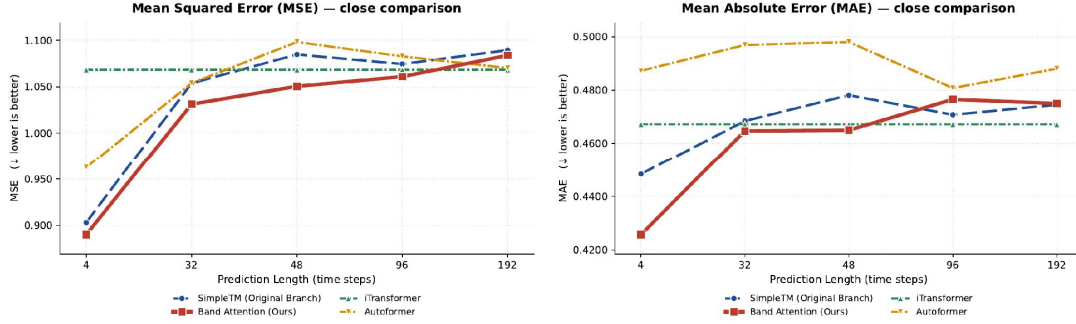


Figure 13: MSE (left) and MAE (right) — close comparison.

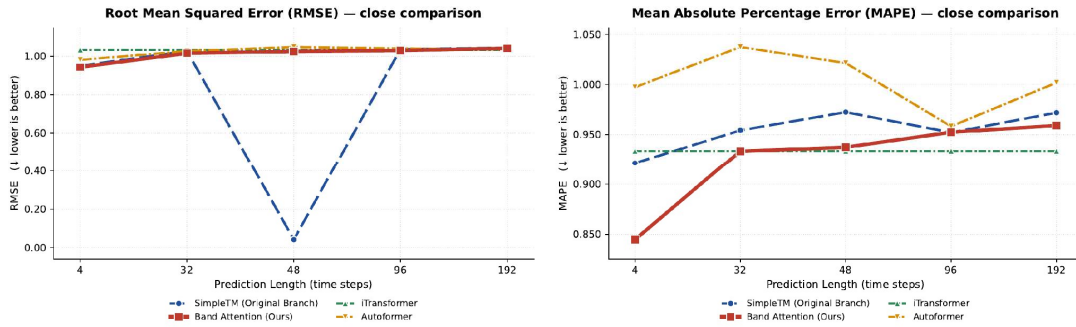


Figure 14: RMSE (left) and MAPE (right) — close comparison.

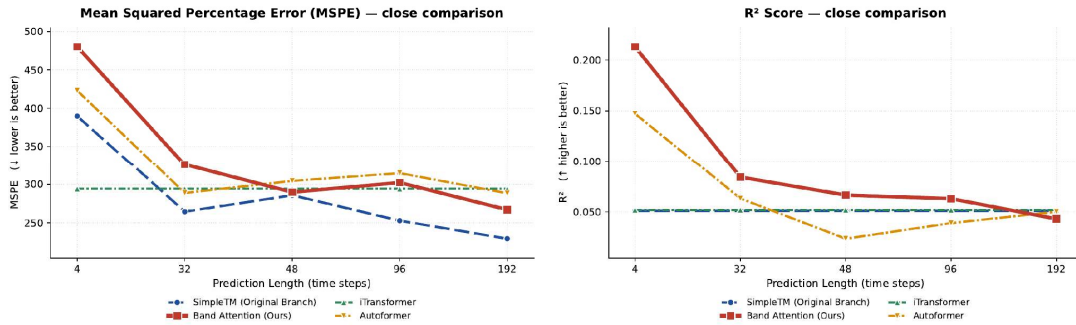


Figure 15: MSPE (left) and R² (right) — close comparison.

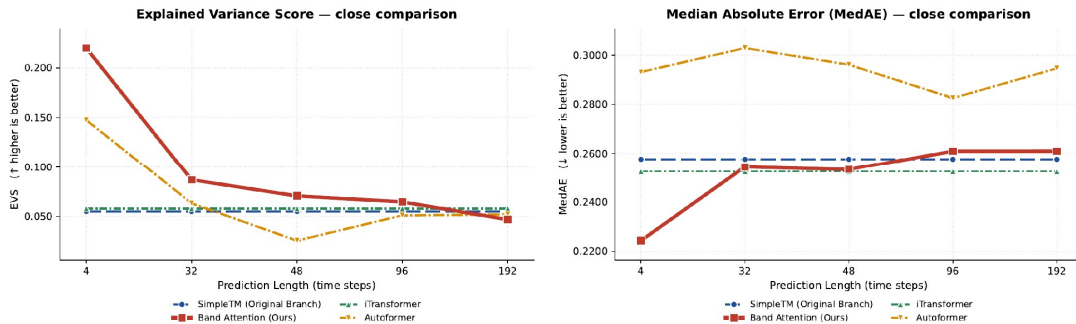


Figure 16: Explained Variance Score (left) and Median Absolute Error (right) — close comparison.

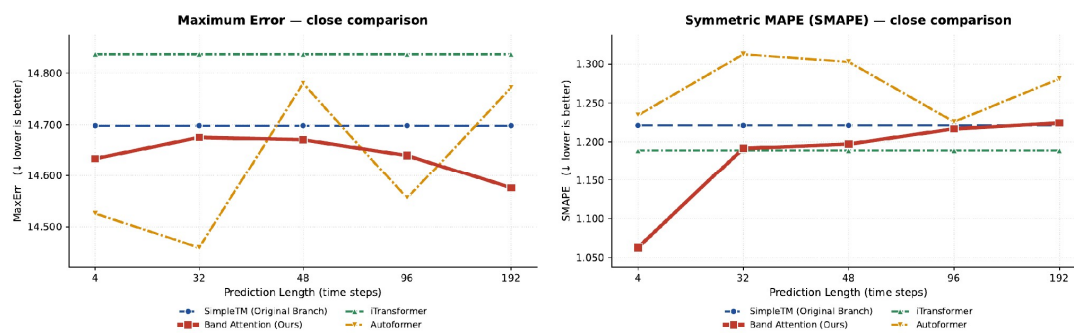


Figure 17: Maximum Error (left) and SMAPE (right) — close comparison.